CYBERSECURITY CAPSTONE PROJECT

PASSWORD STRENGTH CHECKER

A Practical Project Using Python

Submitted by: Aditya Raj Singh

Course: Cybersecurity

Project level: Intermediate

Programming language: Python 3

This project checks password characteristics and gives a basic strength rating to encourage safer password choices.

1. Introduction

Passwords are commonly used to protect online accounts, devices, and personal information. Weak passwords can be easier for attackers to guess. A password strength checker is a simple tool that reviews features of a password, such as its length and the types of characters it contains.

2. Aim of the Project

The aim of this project is to create a Python program that checks a password and classifies it as Weak, Medium, or Strong.

3. Objectives

• Understand why strong passwords are important.

• Check password length and character variety.

• Give users a simple strength result.

• Promote safer password habits.

4. Tools and Technologies

Language: Python 3
Software: Python IDLE, Visual Studio Code, or another Python editor
Libraries: Python built-in string functions only
Hardware: A computer capable of running Python 3.

5. Working of the Project

The user enters a password. The program checks whether it contains at least eight characters, uppercase letters, lowercase letters, digits, and special characters. It then counts the conditions met and displays a strength category with basic advice.

6. Algorithm

• Start the program.

• Ask the user to enter a password.

• Check the password length.

• Check for uppercase and lowercase letters.

• Check for numbers and special characters.

• Calculate the score.

• Display Weak, Medium, or Strong and show advice.

• End the program.

7. Python Source Code

```
import string

def check_password(password):
    score = 0

    if len(password) >= 8:
        score += 1
    if any(ch.isupper() for ch in password):
        score += 1
    if any(ch.islower() for ch in password):
        score += 1
    if any(ch.isdigit() for ch in password):
```

```
        score += 1
    if any(ch in string.punctuation for ch in password):
        score += 1

    if score <= 2:
        return "Weak"
    elif score <= 4:
        return "Medium"
    else:
        return "Strong"


print("PASSWORD STRENGTH CHECKER")
user_password = input("Enter a password to check: ")

result = check_password(user_password)
print("Password strength:", result)

if result == "Weak":
    print("Advice: Use at least 8 characters and mix different character types.")
elif result == "Medium":
    print("Advice: Improve your password by adding missing character types.")
else:
    print("Good! Your password meets all five basic checks.")
```

Privacy note: This simple program checks the password locally while it runs. It does not save or send the password. For real accounts, do not use a password in a public demonstration or share it with anyone.

8. Sample Test Cases

The following are sample/expected test cases to help explain the program. Run the code to confirm the exact output before describing these as completed tests.

Sample input	Expected category	Reason
hello	Weak	Short and limited character variety
Hello123	Medium	Length, upper/lowercase and digits
Hello@12345	Strong	Meets all five checks

9. Advantages

• Easy to use and understand.

• Helps users learn basic password security.

• Runs with standard Python; no extra packages are needed.

• Does not require an internet connection.

10. Limitations

This is an educational checker, not a professional password-security product. It checks only basic character rules. It does not check whether a password has appeared in a data breach, whether it is commonly used, or how resistant it is to modern guessing attacks. A strong rating does not guarantee that a password is secure.

11. Conclusion

The Password Strength Checker demonstrates how Python can be used for a basic cybersecurity task. It checks password length and character variety and provides simple feedback. The project highlights the importance of using long, unique passwords and protecting them carefully.

12. Future Scope

The project could be improved by adding checks for commonly used passwords, a graphical user interface, password phrase guidance, and secure integration with a privacy-preserving breach-checking service.

13. References

1. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
2. National Institute of Standards and Technology (NIST), Digital Identity Guidelines: https://pages.nist.gov/800-63-3/sp800-63b.html